



The interest of productive efficiency and professional training
Misr International Computer & Ai Academy

Smart Home

Hossam Eldein Mohammed
Marwan Gharib Mohammed
Mohammed Emad
Omar Ibrahim Fathi

Under Supervision of
Dr. Michael Nasief
Eng. Hassan Talal

AI Department
February 2025



The interest of productive efficiency and professional training
Misr International Computer & Ai Academy

Smart Home

Hossam Eldein Mohammed
Marwan Gharib Mohammed
Mohammed Emad
Omar Ibrahim Fathi

Under Supervision of
Dr. Michael Nasief
Eng. Hassan Talal

AI Department
February 2025

1. Introduction.....	5
1.1. Project Idea	5
1.1.1. Voice recognition integration	5
1.1.2. Cross-Platform Compatibility	6
1.1.3. Real-Time Control	6
1.2. Project History	6
1.3. Project Objectives	7
1.3.1. Solve	7
1.3.2. Learn	7
1.3.3. Implement	7
1.3.4. Economy	8
1.4. Project Block Diagram	9
2. Scientific Background	10
2.1. Introduction	10
2.2. IOT	12
2.3. Flutter-Based Mobile Application	11
2.4. Firebase Backend	11
2.5. ESP Microcontrollers	12
3. System HW	13
3.1. HW Block diagram	13
3.2. HW Subsystems	14
3.2.1. Relays Circuit	14 - 17
3.2.2. Power Circuit	18- 19
3.2.3. LM2568S	20
3.2.4. MQ-2	21
3.2.5. DHT-22	22
3.2.6. Servo 9g	23
3.2.7. Servo Motor SG-5010	24
4. Embedded SW	25



4.1. SW Flow chart	25
4.2. Function Description	26 - 28
4.3. Complete Code	28
5. Results and Future Work	29
5.1. Final Results	29 - 32
5.2. Conclusion	33
5.3. Future Work	34
6. Complete Code	35 - 65
7. References	66

1. Introduction

1.1: Project Idea:

The project aims to create a Smart Home Control System for easy management of home devices. It focuses on using IoT, and mobile apps. The goal is to provide a seamless, efficient, and secure home environment. This system will integrate advanced technology into everyday life.



Figure 1: Smart home .

Key Futures include:

1.1.1: Voice Recognition Integration:

The system incorporates advanced voice recognition technology, allowing users to issue voice commands to control connected devices such as lights, fans, thermostats, and



Figure 2: Speech to Text.

more. This hands-free operation ensures accessibility and ease of use.

1.1.2 : Cross-Platform Compatibility:

Built using Flutter, the app offers a consistent and responsive experience across both iOS and Android platforms, ensuring broad accessibility.

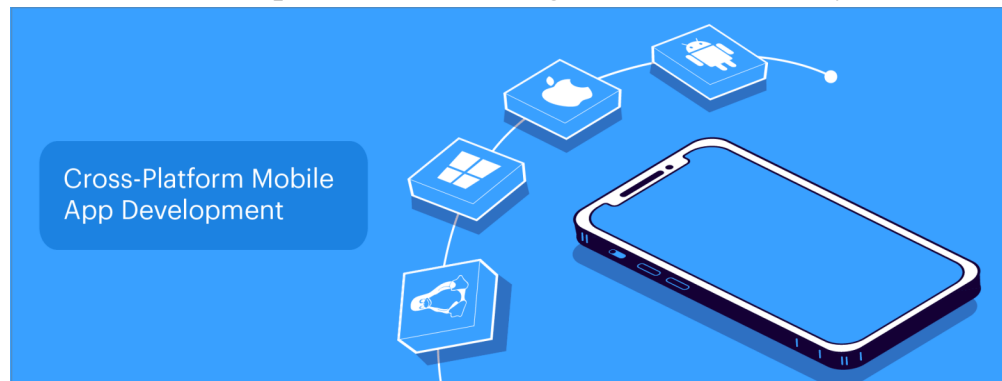


Figure 3: Cross Platform.

1.1.3 : Real-Time Control:

Users can monitor and control their home devices in real-time, whether they are at home or remotely, ensuring security and energy efficiency.

1.2: Project History:

The concept of smart homes began in the early 20th century with visionaries like Nikola Tesla imagining wireless control of devices. In the 1970s, the X10 protocol allowed basic home automation, like remote-controlled lighting. The 2000s brought internet connectivity, enabling remote control via smartphones. By the 2010s, voice assistants like Alexa and Google Assistant made smart homes mainstream, integrating AI and IoT for energy efficiency, security,



Figure 4: smart home.

and convenience. Today, smart homes are a testament to how technology transforms everyday life.

1.3: Project Objectives:

Solve:

Addressing the challenge of managing multiple home devices effectively. Enhancing security, reducing energy wastage, and simplifying home automation. Providing a unified system for easy control of household functions. Ensuring convenience and improved user experience.

Learn:

In this project, we learned:

- 1- ESP programming.
- 2- mobile application programming using Flutter.
- 3- 3D design to model the house.
- 4- We also learned how to use Firebase to store and manage data.

Implement:

Setting up IoT devices, configuring networks, and developing the app. Conducting thorough testing for reliability and user-friendliness. Deploying a comprehensive home automation solution. Aiming for a seamless and user-friendly implementation.



Figure 5: Internet of things (IoT) .

Economy:

N	Product	Quantity	Price	Total price
1	ESP2866	1	275	275
2	Servo 9g	3	100	300
3	Servo 6 Kg.cm	1	350	350
4	Fans	2	50	100
5	Led	3	5	15
6	LM7805	1	10	10
7	LD1117V33	1	20	33
6	Relay 12v	6	12	72
7	ULN 2003	1	12	12
8	LM2596S (3A)	1	75	75
9	MQ-2	1	90	90
10	DHT-22	1	275	275
Cost of materials				1687

Board manufacturing cost (Relays board, Power board)	240
House design cost (wood)	450
The cost of decorating the house	300
Initial cost	2527
Indirect costs (10% of the initial cost)	253
Cost of the final project	2780

1.4: Project Block Diagram:

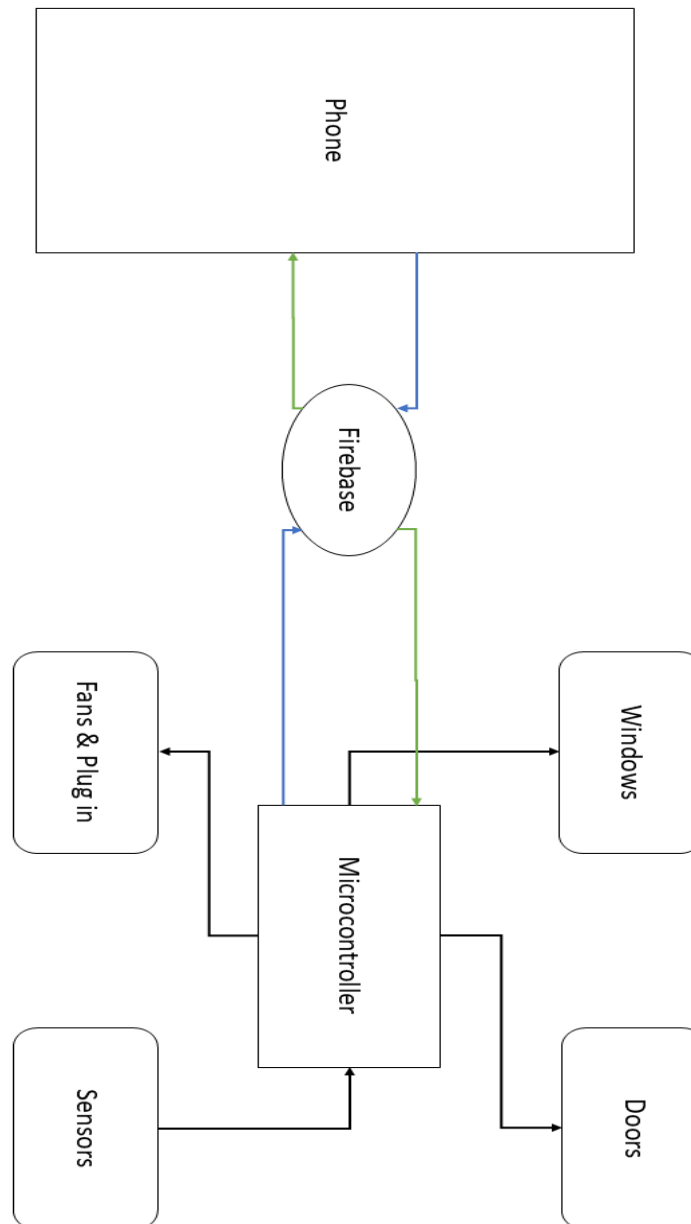


Figure 6: Project Block Diagram .

2. Scientific Background

2.1: Introduction:

Smart Home Automation System that integrates cutting-edge technologies to create a seamless and intelligent living experience. The system consists of a **Flutter-based mobile application**, a **Firestore backend**, and **ESP microcontrollers (ESP8266)** to control and monitor home devices in real time.

2.2: IOT:

The Internet of Things (IoT) is a network of physical devices, vehicles, appliances, and other objects embedded with sensors, software, and connectivity, allowing them to collect and exchange data. In simple terms, IoT is about connecting everyday objects to the internet, enabling them to send and receive information, making our lives smarter and more efficient. For example, a smart thermostat can adjust the temperature based on your habits, or a wearable fitness tracker can monitor your health and sync data to your phone. IoT is all about making devices "talk" to each other to improve convenience, automation, and decision-making.



Figure 7: Iot .

2.3: Flutter-Based Mobile Application:

The **Flutter-based mobile application** serves as the user interface for the smart home system. Built using Google's Flutter framework, the app provides a seamless and responsive experience across both Android and iOS platforms. It features an intuitive design with interactive controls (e.g., buttons, sliders, and switches) to manage smart home devices such as lights, fans, and sensors. The app also displays real-time data (e.g., temperature, humidity) and supports secure user authentication through **Firestore Authentication**. Flutter's flexibility and rich widget library enabled me to create a visually appealing and user-friendly interface.

2.4: Firebase Backend:

The **Firebase backend** acts as the central hub for data storage, synchronization, and communication. Using **Firestore** (or Realtime Database), the app and ESP microcontrollers exchange real-time updates, ensuring that device statuses and sensor data are always in sync. Firebase Authentication ensures secure access to the app, while **Firebase Cloud Messaging (FCM)** enables push notifications for alerts ("Door unlocked"). Firebase's scalability and ease of integration made it the ideal choice for managing the backend of this project.

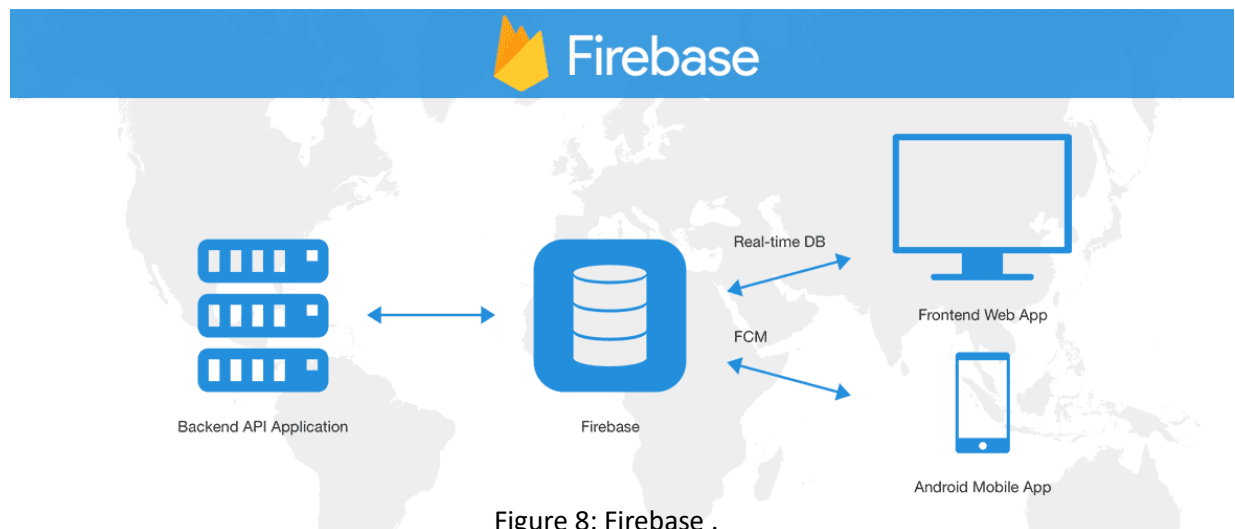


Figure 8: Firebase .

2.5: ESP Microcontrollers:

The **ESP microcontrollers** (ESP8266) are the backbone of the smart home system, connecting physical devices to the digital world. These microcontrollers are programmed to interact with sensors (stemperature) and actuators (relays, LEDs) while communicating with the Firebase backend over Wi-Fi. Using the **Firestore-ESP-Client library**, the ESP devices send sensor data to Firebase and receive commands from the app in real time. This enables features like remote control of devices and real-time monitoring of environmental conditions. The ESP's affordability, versatility, and robust connectivity make it a perfect choice for IoT projects.



Figure 9: ESP8266.

Specification	Value
Clock speed	80MHz
Operating voltage	3.3V
Input Voltage	4.5-10V
Flash Memory	4MB
SRAM	64KB
Digital I/O Pins	11
Analog Pins	1
ADC Range	0-3.3V
UART/SPI/I2C	1/1/1
WiFi Built-In	802.11 b/g/n

3. System HW

3.1: HW Block diagram:

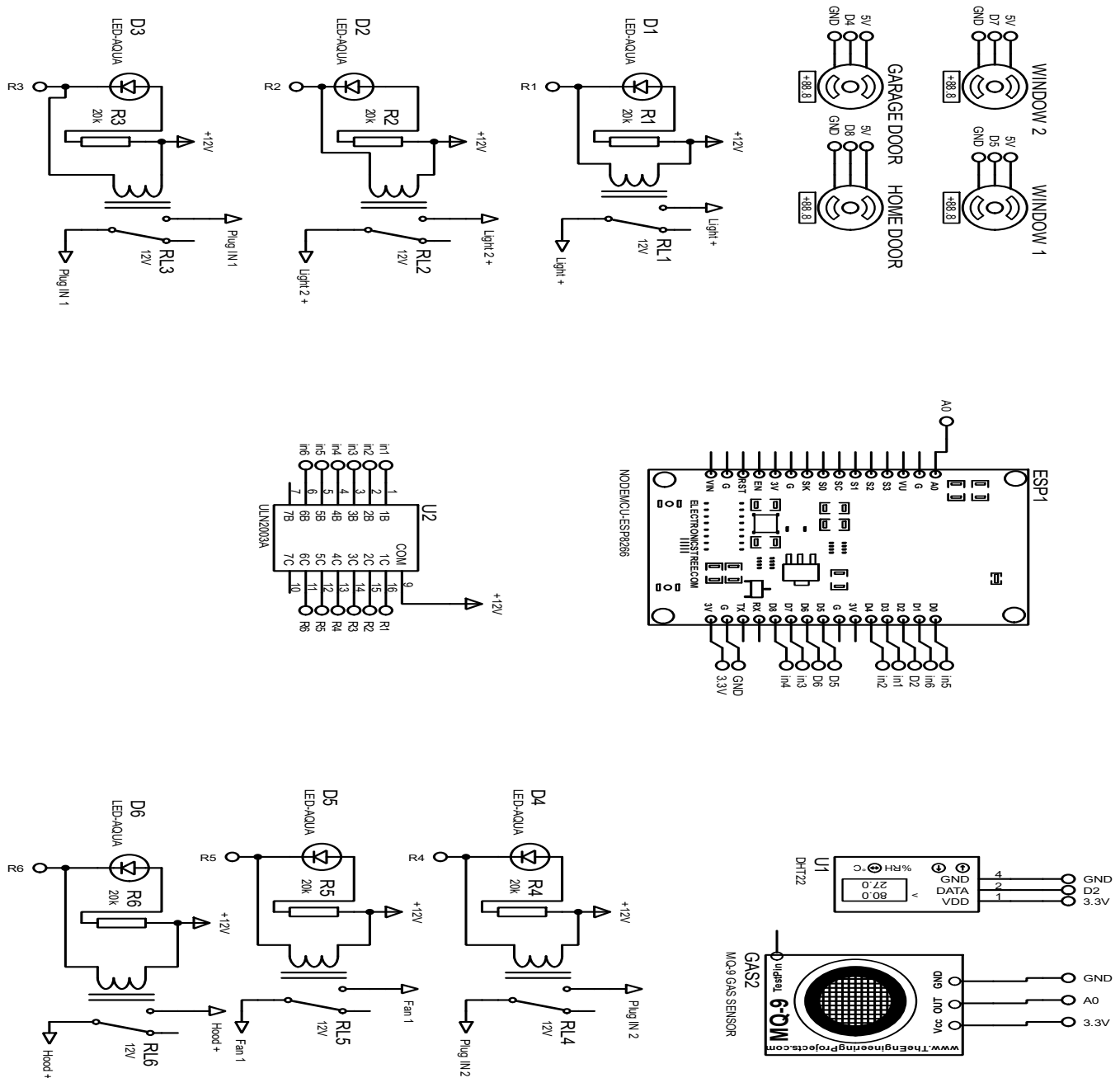


Figure 10: Hardware block diagram.

3.2: HW Subsystems:

1. Relays Circuit.
2. Power Circuit.
3. LM2598S
4. MQ-2.
5. DHT-22.
6. Servo 9g.
7. Servo Motor SG-5010.

3.2.1: Relays Circuit:

❖ Circuit:

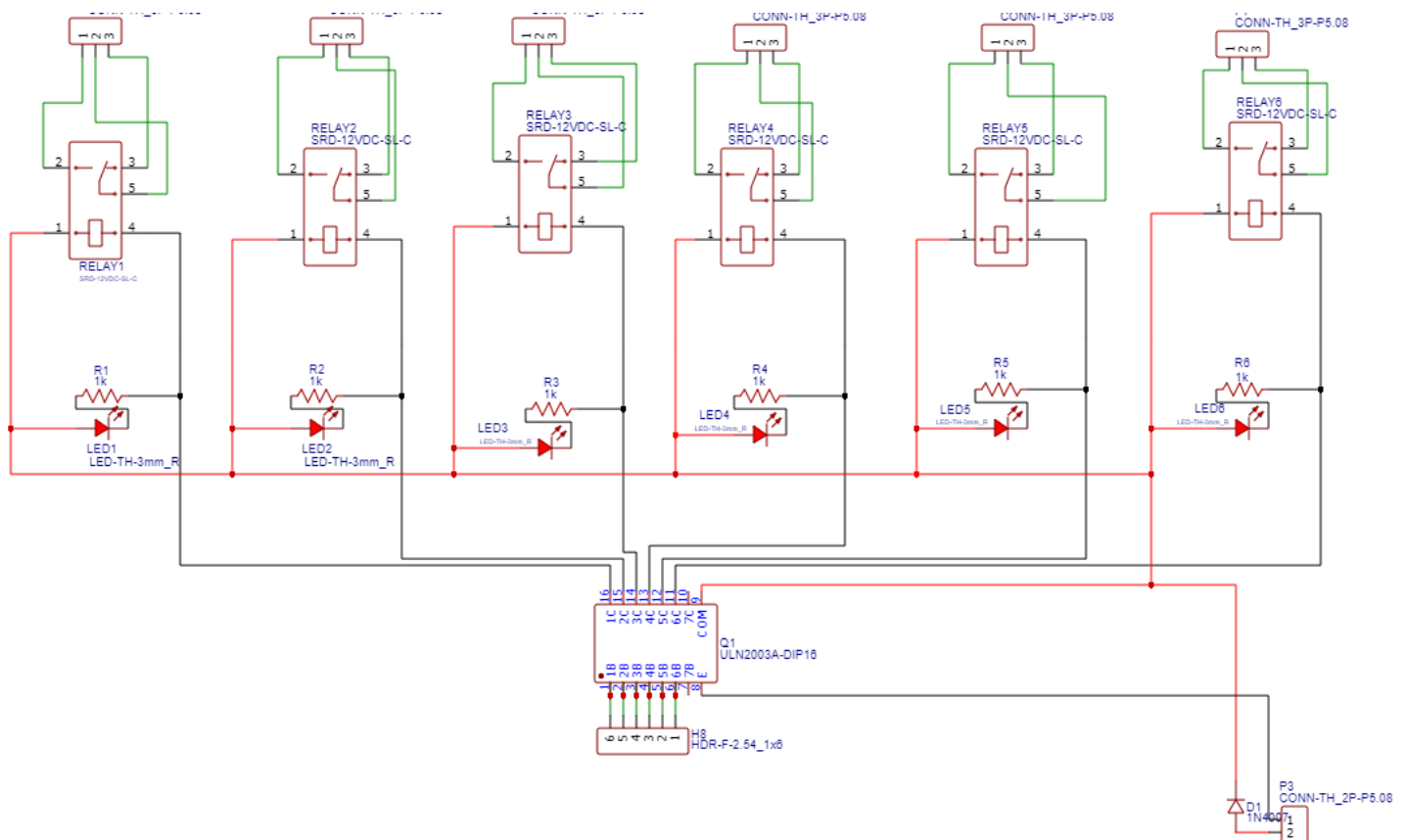


Figure 11: Relays Circuit Schematic(EasyEDA).

❖ PCB Design:

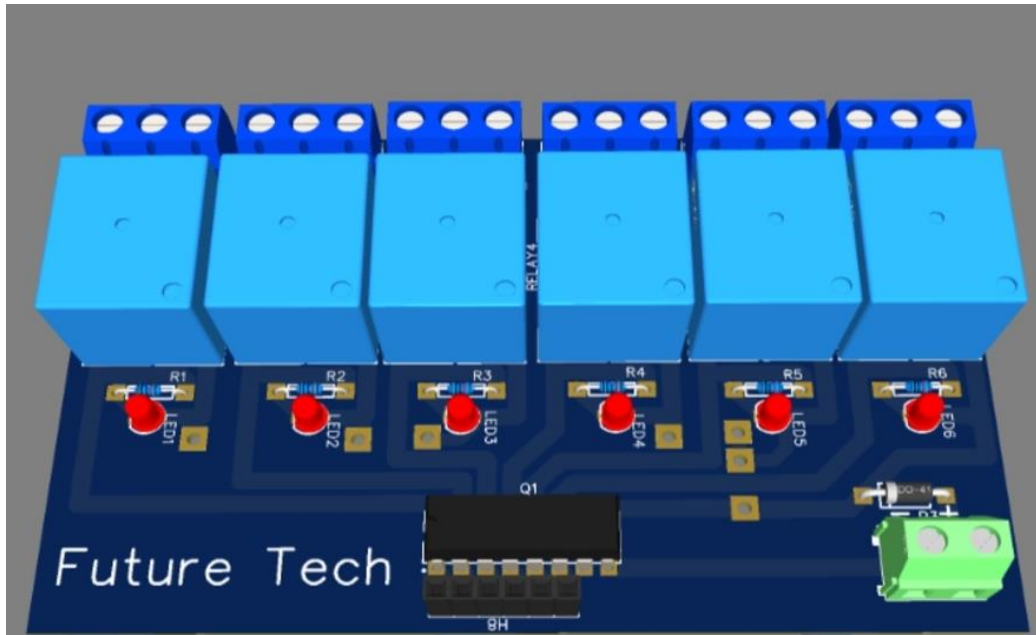


Figure 12: Relays Board Design(EasyEDA).

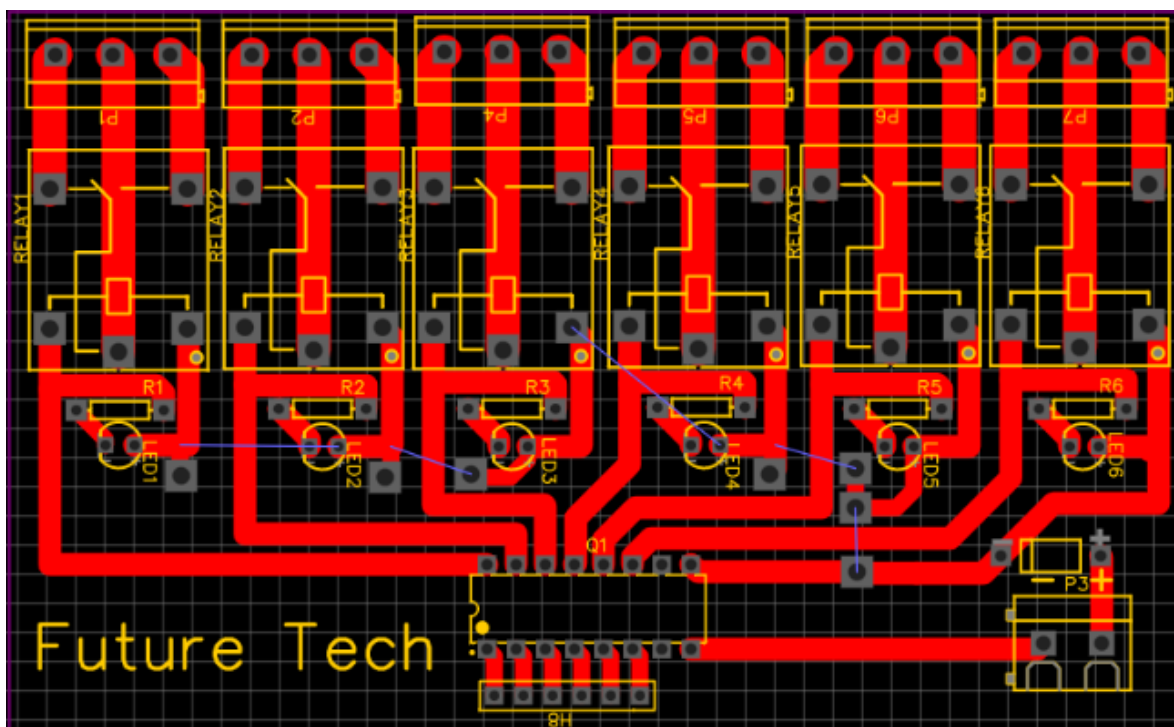


Figure 13: Relays Circuit Paths(EasyEDA).

❖ Description :

This PCB is designed to control multiple relays using the ULN2003, a high-voltage, high-current Darlington transistor array. The ULN2003 acts as an interface between a microcontroller and the relays, ensuring that the relays receive the appropriate current and voltage levels without directly loading the microcontroller's I/O pins.

❖ Data Sheet

- ULN:

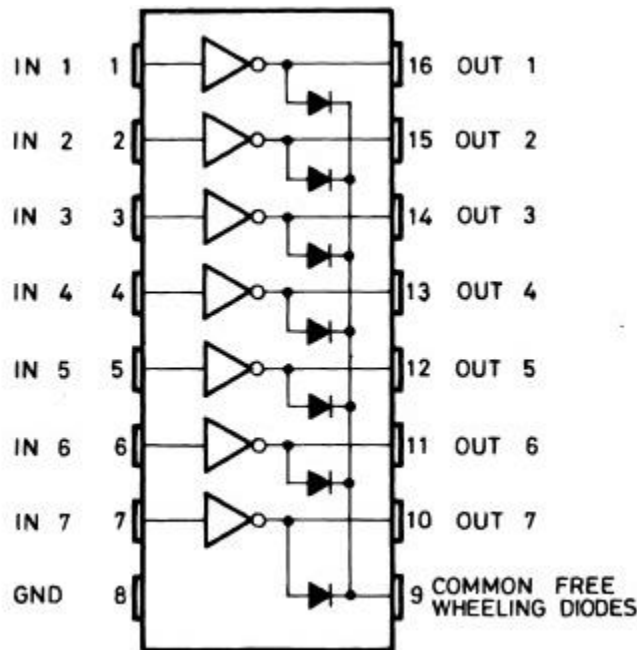


Figure 14: ULN2003.

Parameter	Value	Unit
Output Voltage	50	V
Input Voltage	30	V
Continuous collector current	500	mA
Continuous base current	25	mA
Operating temperature range	-40 to 85	°C

- Relay:

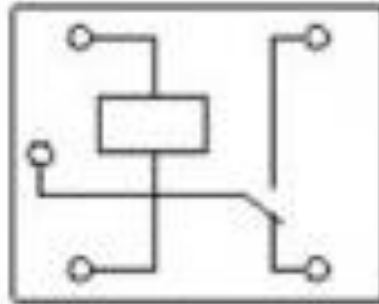


Figure 15: Relay 12V 5-Pins.

Parameter	Value	Unit
Coil Voltage	12	V
Switching Voltage(Max)	250	V
Switching Current(Max)	15	A
Coil Rated Power	360	mW
coil current	30	mA

- Diode :

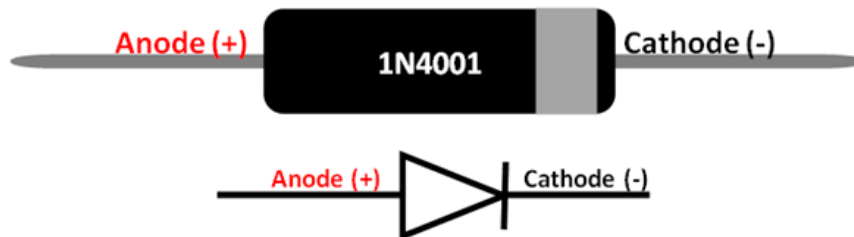


Figure 16: Diode.

Parameter	Value	Unit
Average forward current	1	A
Reverse voltage	35	V
Reverse current	5	uA

3.2.2: Power Circuit Subsystems :

❖ Circuit:

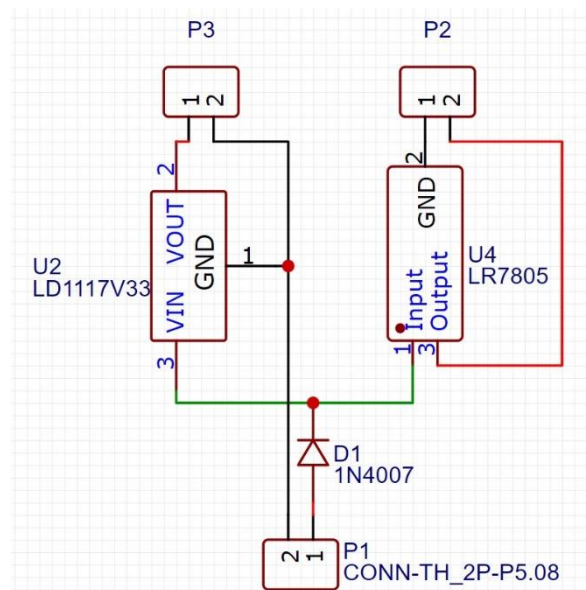


Figure 17: Power Circuit diagram (EasyEDA).

❖ PCB Design:

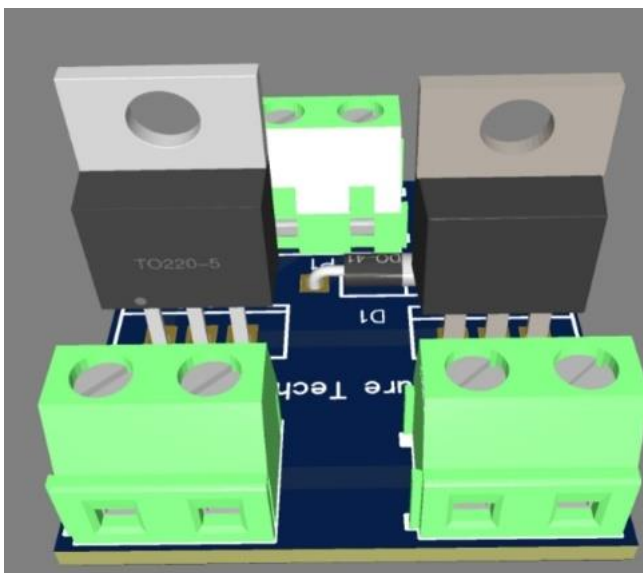


Figure 15: Power Circuit 3D(EasyEDA).

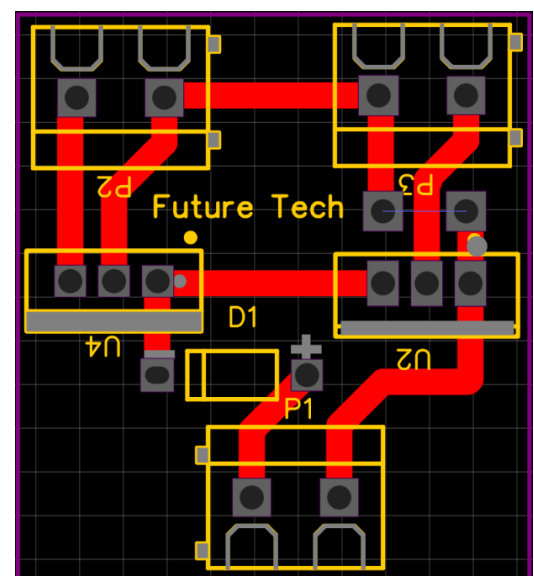


Figure 16: Power Circuit Paths(EasyEDA).

❖ Description :

This PCB is a regulated power supply board designed to convert a 12V DC input into stable 3.3V and 5V outputs, suitable for powering various electronic circuits and microcontrollers.

❖ Datasheet:

- LM7805

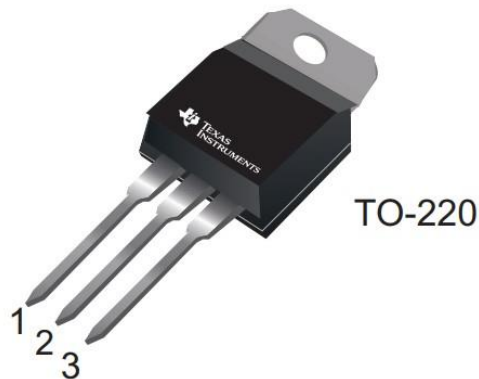


Figure 18: LM7805.

PIN		I/O	DESCRIPTION
NAME	NO.		
INPUT	1	I	Input Voltage Pin
GND	2	I/O	Ground Pin
OUTPUT	3	O	Output Voltage Pin

Paramter	Min	Max	Unit
Output Voltage	4.9	5	V
DC Input Voltage	-	35	V
Output Current	1		A

3.2.3: LM2596S:

❖ Description :

The LM2596 family are monolithic 3-terminal positive voltage regulators designed to provide stable, fixed output voltages. They feature internal current limiting, thermal shutdown, and safe-area compensation, ensuring robust protection and reliability. With adequate heat sinking, they can deliver over 1.5A of output current. These regulators are widely used for local (on-card) voltage regulation to eliminate noise and distribution issues associated with single-point regulation. They can also be adapted for adjustable voltage applications with external components.



Figure 19: LM2596S.

❖ Datasheet:

Paramter	Min	Type	Max	Unit
Output Voltage	4.9	5	5.2	V
DC Input Voltage	-	40		V
Output Current		3		A

3.2.4: MQ-2:

❖ Description :

The MQ-2 sensor is a versatile gas sensor module commonly used for detecting various gases and smoke in the air. It is widely used in safety and monitoring applications due to its ability to sense multiple **gases**, including *flammable gases* and *smoke*.



Figure 20: MQ-2.

❖ Specifications:

Paramter	Value	Unit
Circuit Voltage	5	V
Load resistance	Can adjust	-

3.2.5: DHT-22:

❖ Description :

The **DHT-22** (also known as **AM2302**) is a popular digital *temperature* and *humidity* sensor. It is widely used in environmental monitoring, home automation, and IoT projects due to its accuracy, ease of use, and affordability.



Figure 21: DHT-22.

❖ Specifications:

Paramter	Value
Power supply	3.3-6V DC
Output signal	digital signal via single-bus
Sensing period	2s
Accuracy	humidity +- 2% temperature < +- 0.5

3.2.6: Servo-9g:

❖ Description :

The **9g servo motor** is a small, lightweight, and widely used servo motor in robotics, RC (remote control) applications, and DIY projects. It is known for its compact size, affordability, and ease of control .



Figure 22: DHT-22.

❖ Specifications:

Paramter	Value
Voltage	4.8-6V DC
Speed (Sec)	0.1
Torque (Kg-cm)	2.5
PWM Period	20ms (50Hz)
Degree	180

Note Position "0" (1.5 ms pulse) is middle, "90" (~2ms pulse) is middle, is all the way to the right, "-90" (~1ms pulse) is all the way to the left.

3.2.7: Servo Motor SG-5010:

❖ Description :

The **Servo Motor SG-5010** is a small, lightweight, and widely used servo motor in robotics, RC (remote control) applications, and DIY projects. It is known for its compact size, affordability, and ease of control .



Figure 23: DHT-22.

❖ Specifications:

Paramter	Value
Voltage	4.8-6v
Speed (Sec)	0.19sec/deg (4.8v) 0.15sec/deg (6.0v)
Torque (Kg-cm)	5.5kg/cm(4.8v) 6.5kg/cm(6v)
PWM Period	20ms (50Hz)
Degree	180
Current (idle)	10mA
Current (during movement)	100-300mA
Current (stall)	600mA

4.Embedded SW

4.1: SW Flow Chart:

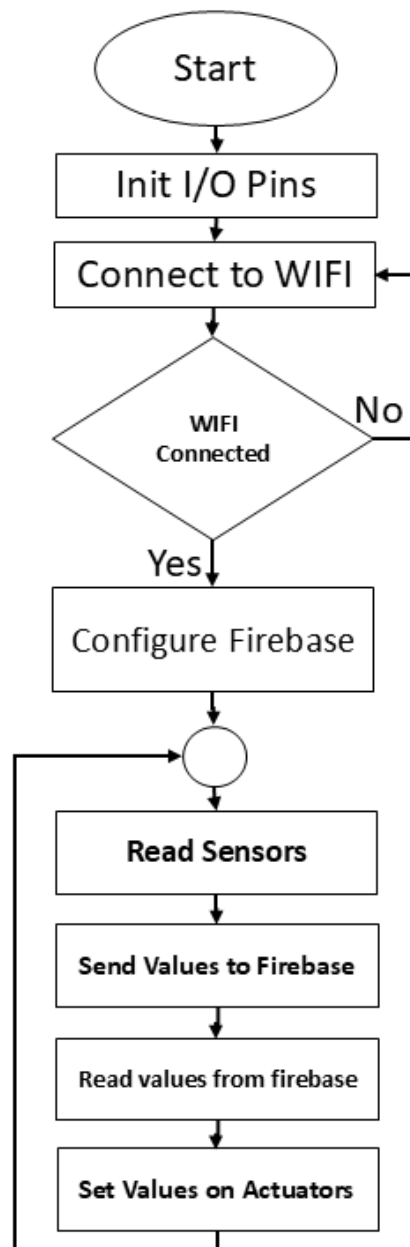


Figure 24: Embedded SW Flow chart.

4.2: Function Description:

```

1 // start and init the dht :
2 dht.begin();
3
4 // Connect to Wi-Fi
5 WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
6
7 // Stuck the program till the wifi connect .
8 while (WiFi.status() != WL_CONNECTED);
9
10 // Configure Firebase
11 config.host = FIREBASE_HOST;
12 config.signer.tokens.legacy_token = FIREBASE_AUTH;
13
14 // Initialize Firebase
15 Firebase.begin(&config, &auth);
16 Firebase.reconnectWiFi(true);

```

Usage	Create object from DHT, Define the firebase Host and Auth
--------------	---

```

1 // Craete Object from the dht .
2 DHT dht(DHT_pin, DHTTYPE);
3 // Firebase project credentials:
4 // Firebase Realtime Database URL
5 #define FIREBASE_HOST "https://smart-home-e3a02-default-rtdb.firebaseio.com/"
6 // Database Auth
7 #define FIREBASE_AUTH "8sqiFHQjQW6f4xTp6koV3VdOgXggqGzgnNlbSiaE"
8

```

Description	1- Init the DHT. 2- Stuck the program until the ESP connect to the WIFI. 3- Init the Fire base and connect to the auth.
--------------------	---



```
1 // Read the Values .
2 mq9_read = (analogRead(Mq9) / 1024.0) * 100.0;
3 temp = dht.readTemperature();
4 humidity = dht.readHumidity();
5
```

Description	Read the sensors value and store it into variables .
--------------------	--



```
1 // read the value of the home led .
2 if (Firebase.getBool(firebaseData, "home_led/status")) {
3   bool home_led = firebaseData.boolData();
4   digitalWrite(homeled, home_led);
5 }
6
```

Description (getBool)

Arguments	Object of Firebase - The Reference to get data it
Return	Boolean (True, False)
Usage	Return true if the Reference is correct and data returned successfully

Description(boolData)

Arguments	Object of Firebase - The Reference to get data it
Return	Boolean (True, False)
Usage	Return the value of Reference



```

1 // Set Values of sensors on the database .
2 Firebase.setFloat(firebaseData, "mq9/value", mq9_read);
3 Firebase.setFloat(firebaseData, "temp/value", temp);
4 Firebase.setFloat(firebaseData, "humidity/value", humidity);
5

```

Description (setFloat)	
Arguments	Object of Firebase - The Reference to set data – Value to set it
Return	No Return
Usage	Set the values of Sensors

4.3: Complete Code:

If you would like to view the code, please refer to the section below.

5. Results And Future Tech

5.1: Final Results:

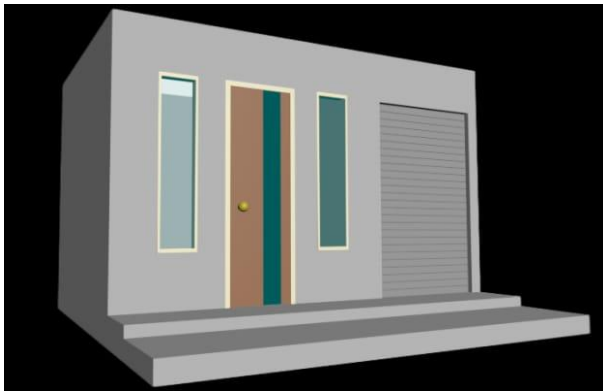


Figure 25: Home makit 3D.

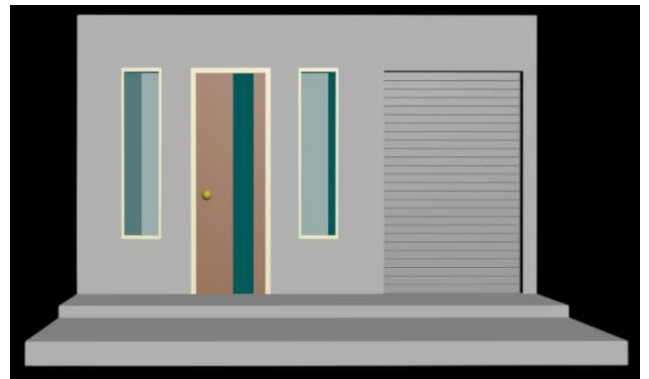


Figure 26: Home makit 3D.

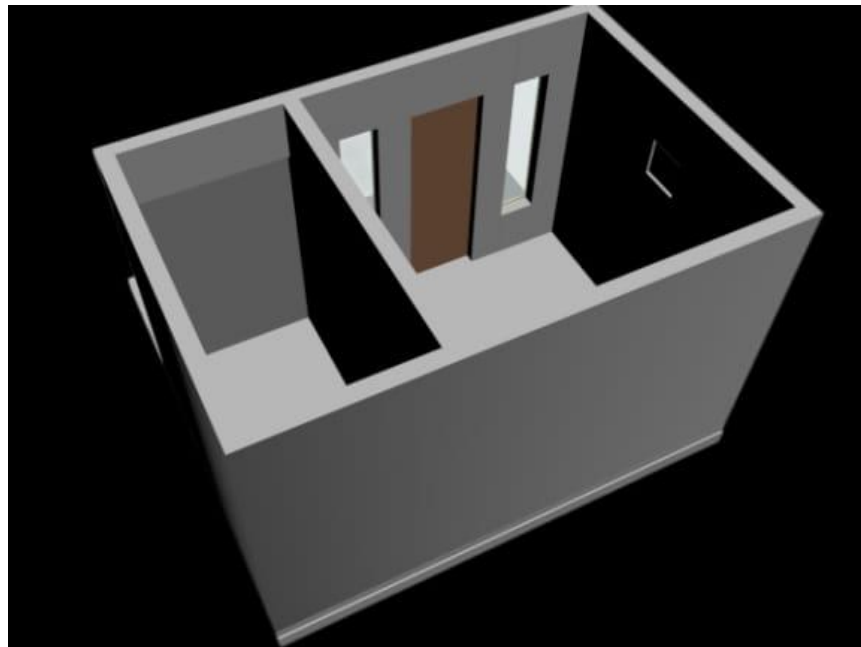


Figure 27: Home makit 3D.



Figure 28: Home Wood.

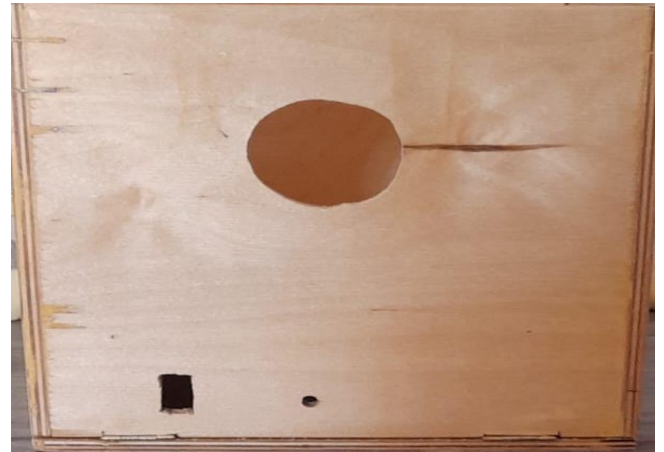


Figure 29: Home Wood.



Figure 30: Home Wood.



Figure 31: Home.



Figure 32: Home.

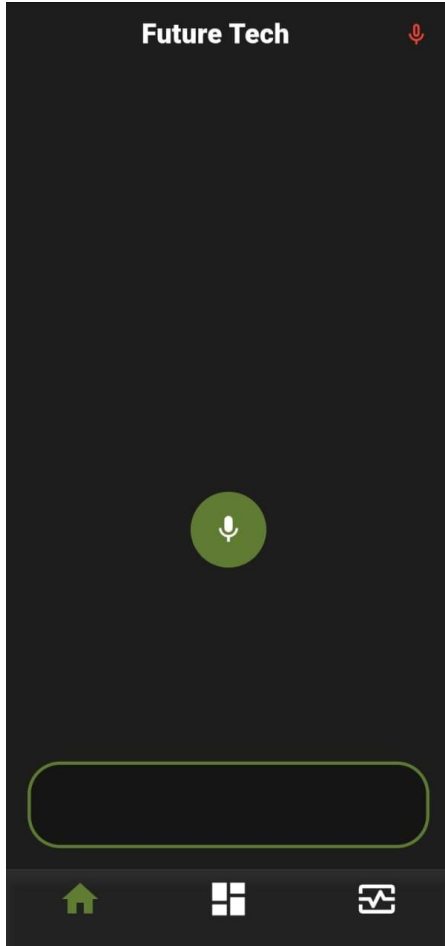


Figure 33: Mic Page.

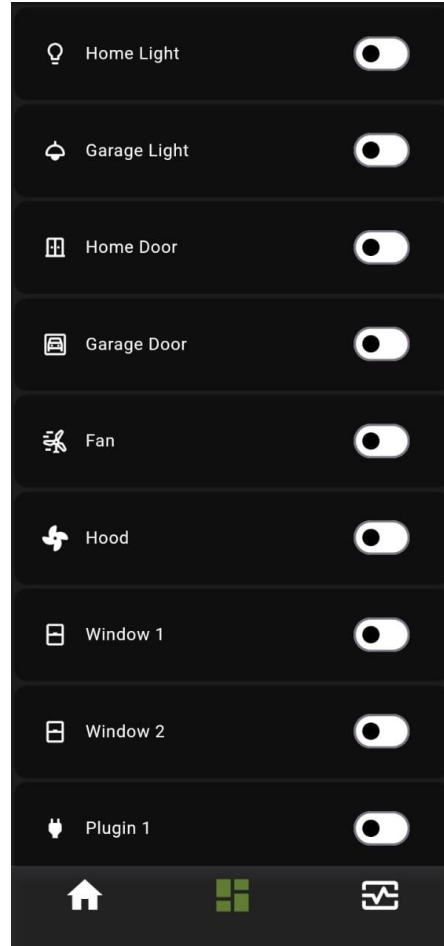


Figure 34: Dashboard.

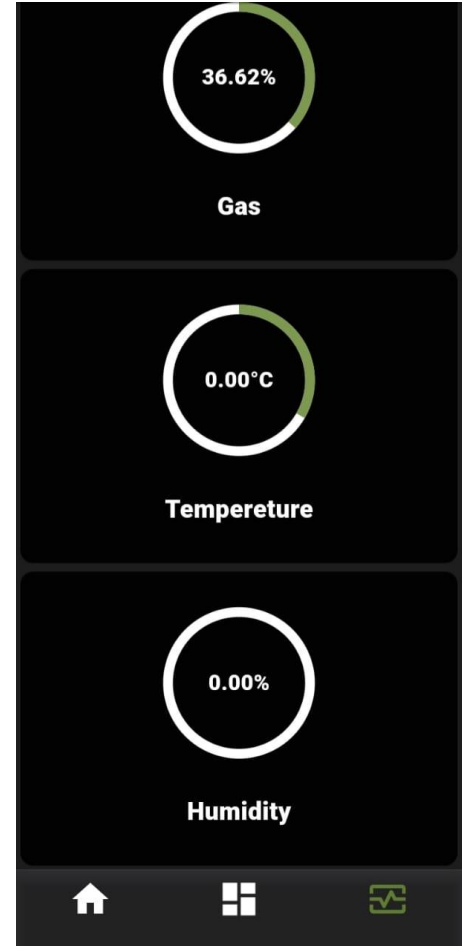


Figure 35: Monitor.

5.2: Conclusion:

As we reach the conclusion of this project, we reflect on the journey that led us to the creation of our Smart Home Control System. This project was not just about designing a functional system but also about expanding our knowledge, applying theoretical concepts to practical applications, and overcoming challenges along the way.

Through our exploration of IoT, mobile application development, and embedded systems, we gained valuable insights into modern technology and its potential to transform everyday life. We tackled complex problems, from integrating hardware components to developing efficient software, ensuring a seamless user experience. Learning to work with ESP programming, Flutter for mobile applications, and Firebase for data management has been an enriching experience that has strengthened our technical expertise.

Beyond the technical aspects, this project emphasized teamwork, problem-solving, and adaptability—skills that are essential in any professional field. The challenges we faced, whether in hardware implementation, software debugging, or system optimization, pushed us to think critically and innovate solutions that enhanced our final product.

The completion of this project is not just an achievement but a stepping stone toward future innovations. Smart home technology continues to evolve, and we hope that our work serves as a foundation for further advancements in automation, security, and energy efficiency.

We extend our deepest gratitude to our supervisors, Dr. Michael and Eng. Hassan Talal, for their guidance and support throughout this journey. Their expertise and encouragement were instrumental in helping us bring this project to fruition. We also appreciate the collaborative spirit among our team members, whose dedication and efforts made this project a success.

In conclusion, this graduation project represents more than just a technical endeavor; it symbolizes our growth, aspirations, and readiness to step into the world of technology and innovation. As we move forward in our careers, we carry with us the lessons, experiences, and confidence gained from this remarkable journey.

5.3: Future Work :

As we look ahead, we aim to expand our work in smart home automation by developing a **Smart Plug-in Device** that can be controlled via a mobile application. This device will be built using the **ESP8266 microchip**, allowing seamless wireless communication and remote control. Our goal is to enhance energy efficiency, improve user convenience, and provide a cost-effective solution for smart home integration.

The smart plug-in will enable users to control various household appliances, monitor power consumption, and even automate schedules for devices, all from a mobile app. By leveraging **IoT technology**, we aim to create a secure, reliable, and easy-to-use system that can be integrated into any home environment.

Beyond this project, we are committed to exploring more **innovative smart solutions** in the future. As technology evolves, we plan to work on **new automation systems, AI-driven enhancements, and advanced IoT applications** that can further improve daily life. Our passion for technology and innovation drives us to continue researching, developing, and implementing new ideas that can contribute to the future of smart living.

This is just the beginning, and we look forward to pushing the boundaries of what is possible in home automation and beyond.

Complete Code

Arduino Code:

```

/*****
*           * Graduation Book *
*   Team Name : * Future Tech *
*   Members :
*       1- Marwan Gharib
*       2- Omar Ibrahim
*       3- Hossam Mohammed
*       4- Mohammed Emad
*****
*           * Under Supvisors *
*       Dr.Michael Naseif
*       Eng.Hassan Talal
*****/

// Include The Library .
#include <ESP8266WiFi.h>
#include <FirebaseESP8266.h>
#include <Servo.h>
#include <DHT.h>

// Craete Objects from servov .
Servo homeservo2;
Servo garageservo;
Servo window servo;
Servo window servo2;

/*****
*   Define the Pins:      *
*****/

// Servos Pins
#define home_servo 14
#define garage_servo 15
#define window_servo 12
#define window2_servo 13

// plug_in && leds
#define homeled 0

```




```
#define garageled 2
#define plug_in1 3
#define plugi_n2 10
// fans
#define fan1 16
#define Hood 5

// MQ
#define Mq9 A0

// DHT22
#define DHT_pin 4
#define DHTTYPE DHT22

// Make the Password and the Username :
#define WIFI_SSID "Marwan"
#define WIFI_PASSWORD "12345678910"

// Craete Object from the dht .
DHT dht(DHT_pin, DHTTYPE);
// Firebase project credentials:
// Firebase Realtime Database URL
#define FIREBASE_HOST "https://smart-home-e3a02-default-rtdb.firebaseio.com/"
// Database Auth
#define FIREBASE_AUTH "8sqiFHQjQW6f4xTp6koV3VdOgxggqGzgnNlbSiaE"

// Create FirebaseConfig and FirebaseAuth objects
FirebaseConfig config;
FirebaseAuth auth;
FirebaseData firebaseData;

// init the variables with value 0 .
float mq9_read = 0, temp = 0, humidity = 0;

// Init the values :
bool GD = false;
bool HD_open = false, HD_close = false;
bool W1_open = false, W1_close = false;
bool W2_open = false, W2_close = false;

void setup() {
    // init the servos and the pins .
    homeservo2.attach(home_servo, 500, 2400);
```



```
garageservo.attach(garage_servo, 500, 2400);
window servo.attach(window_servo, 500, 2400);
window servo2.attach(window2_servo, 500, 2400);

pinMode(homeled, OUTPUT);
pinMode(garageled, OUTPUT);
pinMode(plug_in1, OUTPUT);
pinMode(plug_n2, OUTPUT);

pinMode(fan1, OUTPUT);
pinMode(Hood, OUTPUT);

// start and init the dht :
dht.begin();

// Connect to Wi-Fi
WiFi.begin(WIFI_SSID, WIFI_PASSWORD);

// Stuck the program till the wifi connect .
while (WiFi.status() != WL_CONNECTED);

// Configure Firebase
config.host = FIREBASE_HOST;
config.signer.tokens.legacy_token = FIREBASE_AUTH;

// Initialize Firebase
Firebase.begin(&config, &auth);
Firebase.reconnectWiFi(true);

}

void loop() {

// Read the Values .
mq9_read = (analogRead(Mq9) / 1024.0) * 100.0;
temp = dht.readTemperature();
humidity = dht.readHumidity();

/*****
*           Get the Values from the Firebase           *
*****/

// read the value of the door .
```

```
if (Firebase.getBool(firebaseData, "home_door/status")) {  
    bool home_door = firebaseData.boolData();  
    if (home_door)  
    {  
        if (!HD_open)  
        {  
            for (int i = 13; i <= 120; i+=1)  
            {  
                homeservo2.write(i);  
            }  
        }  
        HD_open = true, HD_close = false;  
    }  
    else  
    {  
        if (!HD_close)  
        {  
            for (int i = 120; i >= 13; i-=1)  
            {  
                homeservo2.write(i);  
            }  
        }  
        HD_open = false, HD_close = true;  
    }  
}  
  
// read the value of the Garage door .  
if (Firebase.getBool(firebaseData, "garage_door/status")) {  
    bool garage_door = firebaseData.boolData();  
    if (garage_door)  
    {  
        if (!GD)  
        {  
            garageservo.write(75);  
            delay(1100);  
        }  
        garageservo.write(180);  
        GD = true;  
    }  
    else  
    {  
        GD = false;  
        garageservo.write(53);  
    }  
}
```

```
}  
}
```

```
// Read the value of window one .  
if (Firebase.getBool(firebaseData, "window_one/status")) {  
    bool window_one = firebaseData.boolData();  
    if (window_one)  
    {  
        if (!W1_open)  
        {  
            for (int i = 10; i <= 130; i+=3)  
            {  
                windowservo.write(i);  
            }  
        }  
        W1_open = true, W1_close = false;  
    }  
    else  
    {  
        if (!W1_close)  
        {  
            for (int i = 130; i >= 10; i-=3)  
            {  
                windowservo.write(i);  
            }  
        }  
        W1_open = false, W1_close = true;  
    }  
}
```

```
// read the value of the window two .  
if (Firebase.getBool(firebaseData, "window_two/status")) {  
    bool window_two = firebaseData.boolData();  
    if (!window_two)  
    {  
        if (!W2_open)  
        {  
            for (int i = 0; i <= 125; i += 3)  
            {  
                windowservo2.write(i);  
            }  
        }  
        W2_open = true, W2_close = false;
```

```
}  
else  
{  
  if (!W2_close)  
  {  
    for (int i = 125; i >= 0; i -= 3)  
    {  
      windowservo2.write(i);  
    }  
  }  
  W2_open = false, W2_close = true;  
}  
}  
  
// read the value of the home led .  
if (Firebase.getBool(firebaseData, "home_led/status")) {  
  bool home_led = firebaseData.boolData();  
  digitalWrite(homeled, home_led);  
}  
  
// Read the Value of garage led .  
if (Firebase.getBool(firebaseData, "garage_led/status")) {  
  bool garage_led = firebaseData.boolData();  
  digitalWrite(garageled, garage_led);  
}  
  
// Read the value of fan .  
if (Firebase.getBool(firebaseData, "fan/status")) {  
  bool fan = firebaseData.boolData();  
  digitalWrite(fan1, fan);  
}  
  
// Read the value of hood .  
if (Firebase.getBool(firebaseData, "hood/status")) {  
  bool hood = firebaseData.boolData();  
  digitalWrite(Hood, hood);  
}  
  
// Read the value of plug in one .  
if (Firebase.getBool(firebaseData, "plug_in_one/status")) {  
  bool plug_in_one = firebaseData.boolData();  
  digitalWrite(plug_in1, plug_in_one);  
}
```

}

// Read the value of the plug in two .

```
if (Firebase.getBool(firebaseData, "plug_in_two/status")) {  
    bool plug_in_two = firebaseData.boolData();  
    digitalWrite(plugi_n2, plug_in_two);  
}
```

// Set Values of sensors on the database .

```
Firebase.setFloat(firebaseData, "mq9/value", mq9_read);  
Firebase.setFloat(firebaseData, "temp/value", temp);  
Firebase.setFloat(firebaseData, "humidity/value", humidity);
```

}



Flutter Code:

/******

* * Graduation Book * *

* Team Name : * Future Tech * *

* Members : *

* 1- Marwan Gharib *

* 2- Omar Ibrahim *

* 3- Hossam Mohammed *

* 4- Mohammed Emad *

* * Under Supervisors * *

* Dr.Michael Naseif *

* Eng.Hassan Talal *

*****/

// Import the libraries :

import "package:flutter/material.dart";

import 'package:firebase_core/firebase_core.dart';

import 'commands.dart';

import 'dashBoard.dart';

import 'monitor.dart';

// Craete Widgets from value notifier .

ValueNotifier<bool> isListeningNotifier = ValueNotifier<bool>(false);

void main() async {

/**



* Init the Firebase .

*/

```
WidgetsFlutterBinding.ensureInitialized();
```

```
await Firebase.initializeApp();
```

```
runApp(const MyApp());
```

```
}
```

```
class MyApp extends StatefulWidget {
```

```
  const MyApp({super.key});
```

```
  @override
```

```
  State<MyApp> createState() => _MyApplication();
```

```
}
```

```
class _MyApplication extends State<MyApp> {
```

```
  // init the variable for indexing .
```

```
  int indexBar = 0;
```

```
  // Create list from the widgets .
```

```
  List<Widget> app = const [Commands(), Dashboard(), Monitor()];
```

```
  @override
```

```
  Widget build(BuildContext context) {
```

```
    return MaterialApp(
```

```
      debugShowCheckedModeBanner: false,
```

```
      home: Scaffold(
```

```
        bottomNavigationBar: BottomNavigationBar(
```



```
currentIndex: indexBar,

onTap: (value) {
  setState() {
    indexBar = value;
  });
},

fixedColor: const Color.fromARGB(221, 106, 139, 53),
backgroundColor: const Color.fromARGB(207, 0, 0, 0),
unselectedItemColor: Colors.white,
items: const [
  BottomNavigationBarItem(
    icon: Icon(
      Icons.home,
      size: 40,
    ),
    label: ")",
  BottomNavigationBarItem(
    icon: Icon(
      Icons.dashboard,
      size: 40,
    ),
    label: ")",
  BottomNavigationBarItem(
    icon: Icon(
      Icons.monitor_heart_outlined,
      size: 40,
    ),
    label: ")",
```

label: ")

]),

body: Container(

color: const Color.fromARGB(225, 0, 0, 0),

child: app.elementAt(indexBar)),

));

}

}

// Import the libraries :

import 'package:flutter/material.dart';

import 'package:firebase_database/firebase_database.dart';

import 'package:font_awesome_flutter/font_awesome_flutter.dart';

class Dashboard extends StatefulWidget {

const Dashboard({super.key});

@override

State<Dashboard> createState() => _DashboardState();

}

class _DashboardState extends State<Dashboard> {

// Listen to all values of the Fire Base and store it:

@override

void initState() {

super.initState();

```
for (int i = 0; i < refs.length; i++) {  
  refs[i].onValue.listen((event) {  
    final data = event.snapshot.value;  
    if (data is bool) {  
      setState() {  
        statuses[i] = data; // Update the switch state  
      });  
    }  
  });  
}  
}  
  
@override  
Widget build(BuildContext context) {  
  // Create object from the custom widgets .  
  return ListView.builder(  
    itemCount: myCards.length,  
    itemBuilder: (context, index) => myCards[index]);  
}  
  
// ignore: must_be_immutable  
class Cards extends StatefulWidget {  
  Icon? icon;  
  String? title;  
  int? index;
```



Cards(

```
{super.key,  
  required this.icon,  
  required this.title,  
  required this.index});
```

@override

```
// ignore: no_logic_in_create_state
```

```
State<Cards> createState() =>
```

```
  // ignore: no_logic_in_create_state
```

```
  _CardsState(icon: icon, title: title, index: index);
```

```
}
```

```
class _CardsState extends State<Cards> {
```

```
  // Craete the class .
```

```
  // Pramaters :
```

```
  Icon? icon;
```

```
  String? title;
```

```
  int? index;
```

```
  // Init the class constructor .
```

```
  _CardsState({required this.icon, required this.title, required this.index});
```

@override

```
Widget build(BuildContext context) {
```

```
  return Card(
```

```
    margin: const EdgeInsets.fromLTRB(3, 2, 3, 2),
```

```

color: const Color.fromARGB(80, 0, 0, 0),
child: Padding(
  padding: const EdgeInsets.fromLTRB(10, 15, 10, 15),
  child: ListTile(
    leading: icon,
    title: Text(
      title!,
      style: const TextStyle(color: Colors.white),
    ),
    trailing: Switch(
      activeTrackColor: const Color.fromARGB(221, 106, 139, 53),
      inactiveThumbColor: Colors.black,
      inactiveTrackColor: Colors.white,
      value: statuses[index!],
      onChanged: (val) {
        setState(() {
          statuses[index!] = val;
          _turnObject(index!, statuses[index!]);
        });
      },
    ),
  ),
);
}

```

// This function to turn the object .

```

void _turnObject(int index, bool status) {

```

```
try {  
    refs[index].set(status).catchError((error) {});  
    // ignore: empty_catches  
} catch (e) {}  
}  
}  
  
// list ro store the objects state :  
List<bool> statuses = [  
    false,  
    false,  
    false,  
    false,  
    false,  
    false,  
    false,  
    false,  
    false,  
    false,  
    false  
];  
  
// list of References :  
final List<DatabaseReference> refs = [  
    FirebaseDatabase.instance.ref('home_led/status'),  
    FirebaseDatabase.instance.ref('garage_led/status'),  
    FirebaseDatabase.instance.ref('home_door/status'),  
    FirebaseDatabase.instance.ref('garage_door/status'),
```

```
FirebaseDatabase.instance.ref('fan/status'),
FirebaseDatabase.instance.ref('hood/status'),
FirebaseDatabase.instance.ref('window_one/status'),
FirebaseDatabase.instance.ref('window_two/status'),
FirebaseDatabase.instance.ref('plug_in_one/status'),
FirebaseDatabase.instance.ref('plug_in_two/status'),
];

// List of widgits :
List<Cards> myCards = [
  Cards(
    icon: const Icon(Icons.lightbulb_outline, color: Colors.white),
    title: "Home Light",
    index: 0,
  ),
  Cards(
    icon: const Icon(Icons.light, color: Colors.white),
    title: "Garage Light",
    index: 1,
  ),
  Cards(
    icon: const Icon(Icons.door_sliding_outlined, color: Colors.white),
    title: "Home Door",
    index: 2,
  ),
  Cards(
    icon: const Icon(Icons.garage_outlined, color: Colors.white),
```

title: "Garage Door",

index: 3,

),

Cards(

icon: const Icon(Icons.wind_power_outlined, color: Colors.white),

title: "Fan",

index: 4,

),

Cards(

icon: const Icon(FontAwesomeIcons.fan, color: Colors.white),

title: "Hood",

index: 5,

),

Cards(

icon: const Icon(Icons.sensor_window_outlined, color: Colors.white),

title: "Window 1",

index: 6,

),

Cards(

icon: const Icon(Icons.sensor_window_outlined, color: Colors.white),

title: "Window 2",

index: 7,

),

Cards(

icon: const Icon(Icons.power, color: Colors.white),

title: "Plugin 1",

index: 8,


```
),  
Cards(  
  icon: const Icon(Icons.power, color: Colors.white),  
  title: "Plugin 2",  
  index: 9,  
)  
];
```

```
// Import libraries :  
import 'package:flutter/material.dart';  
import 'package:speech_to_text/speech_to_text.dart';  
import 'package:firebase_database/firebase_database.dart';  
import 'main.dart';  
import 'dashBoard.dart';  
  
class Commands extends StatefulWidget {  
  const Commands({super.key});  
  
  @override  
  State<Commands> createState() => TextVoiceCommand();  
}  
  
class TextVoiceCommand extends State<Commands> {  
  // Create Objects .  
  GlobalKey<FormState> key = GlobalKey();  
  SpeechToText speech = SpeechToText();  
  TextEditingController x = TextEditingController();
```



```
// Craete variables
```

```
bool isInitialized = false;
```

```
String recognized = "";
```

```
int? index;
```

```
double m = 70;
```

```
@override
```

```
Widget build(BuildContext context) {
```

```
  return Column(
```

```
    mainAxisAlignment: MainAxisAlignment.spaceBetween,
```

```
    children: [
```

```
      Padding(
```

```
        padding: const EdgeInsets.fromLTRB(20, 50, 20, 0),
```

```
        child: Row(
```

```
          mainAxisAlignment: MainAxisAlignment.spaceBetween,
```

```
          children: [
```

```
            const Text(""),
```

```
            const Text(
```

```
              "Future Tech",
```

```
              style: TextStyle(
```

```
                fontSize: 25,
```

```
                fontWeight: FontWeight.w900,
```

```
                color: Colors.white),
```

```
            ),
```

```
            Icon(
```

```
              Icons.mic_none_outlined,
```

```
color: isListeningNotifier.value ? Colors.white : Colors.red,  
    )  
  ],  
),  
),  
displayCommand(),  
mic(),  
Form(  
  key: key,  
  child: Padding(  
    padding: const EdgeInsets.all(20),  
    child: Row(  
      children: [textFeild()],  
    ),  
  ),  
),  
),  
],  
);  
}
```

// This functio to listen the values of objects .

```
@override  
void initState() {  
  super.initState();  
  initSpeechToText();  
  for (int i = 0; i < refs.length; i++) {  
    refs[i].onValue.listen((event) {});  
  }
```

```
}
```

```
}
```

```
// Init the speech to text.
```

```
Future<void> initSpeechToText() async {
```

```
  bool available = await speech.initialize();
```

```
  if (available) {
```

```
    setState() {
```

```
      isInitialized = true;
```

```
    });
```

```
  }
```

```
}
```

```
// Function to start the Listen from mic .
```

```
void startListening() {
```

```
  if (isInitialized) {
```

```
    speech.listen(
```

```
      onResult: (result) {
```

```
        setState() {
```

```
          recognized = result.recognizedWords;
```

```
        });
```

```
      },
```

```
      localeId: 'en_US');
```

```
  if (mounted) {
```

```
    setState() {
```

```
      isListeningNotifier.value = true;
```

```
    });
```

```
}  
}  
}  
  
// Function to stop the listen of mic .  
void stopListening() {  
    if (isListeningNotifier.value) {  
        speech.stop();  
        setState() {  
            isListeningNotifier.value = false;  
        });  
    }  
}
```

```
// Turn on object :  
void turnOn(int index) {  
    try {  
        refs[index].set(true).catchError((error) {});  
        // ignore: empty_catches  
    } catch (e) {}  
}
```

```
// turn off the objects :  
void turnOff(int index) {  
    try {  
        refs[index].set(false).catchError((error) {});  
        // ignore: empty_catches
```

```
} catch (e) {}
```

```
}
```

```
// Seed command if false or true to database :
```

```
void sendCommand(String command) {
```

```
// If the commands if found :
```

```
if (commandsOn.contains(command.toLowerCase())) {
```

```
    index = commandsOn.indexOf(command.toLowerCase());
```

```
    if (index != null) {
```

```
        statuses[index!] = true;
```

```
        // Turn On the Object .
```

```
        turnOn(index!);
```

```
    }
```

```
} else if (commandsOff.contains(command.toLowerCase())) {
```

```
    index = commandsOff.indexOf(command.toLowerCase());
```

```
    if (index != null) {
```

```
        statuses[index!] = false;
```

```
        // Turn Off the Object .
```

```
        turnOff(index!);
```

```
    }
```

```
}
```

```
}
```

```
Expanded textFeild() {
```

```
    return Expanded(
```

```
        child: TextFormField(
```

```
            cursorColor: Colors.black,
```

```
validator: (value) {  
  if (value!.isEmpty ||  
    commandsOn.contains(value) ||  
    commandsOff.contains(value)) {  
    return null;  
  }  
  return "Wrong Command";  
},  
onChanged: (value) {  
  setState() {  
    key.currentState!.validate();  
  });  
},  
controller: x,  
decoration: InputDecoration(  
  fillColor: const Color.fromARGB(69, 0, 0, 0),  
  filled: true,  
  border: const OutlineInputBorder(),  
  focusedBorder: OutlineInputBorder(  
    borderRadius: BorderRadius.circular(30),  
    borderSide: const BorderSide(color: Colors.white, width: 3),  
  ),  
  enabledBorder: OutlineInputBorder(  
    borderRadius: BorderRadius.circular(30),  
    borderSide: const BorderSide(  
      color: Color.fromARGB(221, 106, 139, 53),  
      width: 3,
```

```
),  
,  
suffix: send()),  
style: const TextStyle(fontSize: 18, color: Colors.white)),  
);  
}  
  
Text displayCommand() {  
  return Text(  
    "\n ${recognized.toLowerCase()}",  
    style: TextStyle(  
      fontSize: 30,  
      color: commandsOn.contains(recognized.toLowerCase()) ||  
        commandsOff.contains(recognized.toLowerCase())  
        ? Colors.white  
        : Colors.red),  
  );  
}  
  
Widget mic() {  
  return GestureDetector(  
    onTapDown: (details) {  
      m = 90;  
      startListening();  
    },  
    onTapUp: (details) {  
      m = 70;
```



```
stopListening();  
sendCommand(recognized);  
},  
child: Container(  
  height: m,  
  width: m,  
  decoration: BoxDecoration(  
    color: const Color.fromARGB(221, 106, 139, 53),  
    borderRadius: BorderRadius.circular(50)),  
  child: const Icon(  
    Icons.mic,  
    color: Colors.white,  
    size: 30,  
  ),  
,  
);  
}  
  
Widget send() {  
  return IconButton(  
    onPressed: () => sendCommand(x.text),  
    icon: const Icon(  
      Icons.send,  
      color: Colors.white,  
      size: 30,  
    ));  
}
```

}

// Commands to open the objects .

```
List<String> commandsOn = [
```

```
    "open light of room",
```

```
    "open light of garage",
```

```
    "open door of room",
```

```
    "open door of garage",
```

```
    "open fan",
```

```
    "open hood",
```

```
    "open window 1",
```

```
    "open window 2",
```

```
    "connect plugin 1",
```

```
    "connect plugin 2"
```

```
];
```

// Commands to close the objects .

```
List<String> commandsOff = [
```

```
    "close light of room",
```

```
    "close light of garage",
```

```
    "close door of room",
```

```
    "close door of garage",
```

```
    "close fan",
```

```
    "close hood",
```

```
    "close window 1",
```

```
    "close window 2",
```

```
    "disconnect plugin 1",
```

"disconnect plugin 2"

];

// Ref for the database .

```
final List<DatabaseReference> refs = [  
  FirebaseDatabase.instance.ref('home_led/status'),  
  FirebaseDatabase.instance.ref('garage_led/status'),  
  FirebaseDatabase.instance.ref('home_door/status'),  
  FirebaseDatabase.instance.ref('garage_door/status'),  
  FirebaseDatabase.instance.ref('fan/status'),  
  FirebaseDatabase.instance.ref('hood/status'),  
  FirebaseDatabase.instance.ref('window_one/status'),  
  FirebaseDatabase.instance.ref('window_two/status'),  
  FirebaseDatabase.instance.ref('plug_in_one/status'),  
  FirebaseDatabase.instance.ref('plug_in_two/status'),  
];
```

```
import 'package:flutter/material.dart';  
import 'package:percent_indicator/circular_percent_indicator.dart';  
import 'package:firebase_database/firebase_database.dart';
```

// Create list to store the values.

```
List<String> analogValues = ["0", "0", "0"];
```

```
class Monitor extends StatefulWidget {
```

```
  const Monitor({super.key});
```

```
  @override
```

```
State<Monitor> createState() => _MonitorDisplay();  
}  
  
class _MonitorDisplay extends State<Monitor> {  
  @override  
  Widget build(BuildContext context) {  
    return Column(  
      children: [  
        Custom("Gas", (double.parse(analogValues[0])) / 100, '%'),  
        Custom("Tempereture", (((double.parse(analogValues[1])) + 40) / 120),  
          "°C"),  
        Custom("Humidity", (double.parse(analogValues[2])) / 100, '%'),  
      ],  
    );  
  }  
  
  @override  
  void initState() {  
    super.initState();  
    _setupRealtimeListener(); // Call the function to get the data when the widget is initialized  
  }  
  
  // Read the data from the firebase .  
  void _setupRealtimeListener() {  
    // Listen to the value changes in Firebase in real-time  
    for (int i = 0; i < refs.length; i++) {  
      refs[i].onValue.listen((DatabaseEvent event) {
```

```

final data = event.snapshot.value;

setState() {
  analogValues[i] = data.toString();
});
});
}
}
}

Expanded Custom(String text, double percent, String mark) {
  return Expanded(
    child: SizedBox(
      width: 900,
      child: Card(
        color: const Color.fromARGB(225, 0, 0, 0),
        child: Column(
          mainAxisAlignment: MainAxisAlignment.spaceEvenly,
          children: [
            CircularPercentIndicator(
              radius: 70,
              lineWidth: 9,
              percent: percent,
              center: Text(
                "${mark == '°C' ? (double.parse(analogValues[1])).toStringAsFixed(2) : (percent *
100).toStringAsFixed(2)}$mark",
                style: const TextStyle(
                  fontWeight: FontWeight.w900,

```



```
fontSize: 20,
color: Colors.white),
),
progressColor: const Color.fromARGB(221, 106, 139, 53),
backgroundColor: const Color.fromARGB(255, 255, 255, 255),
),
Text(
text,
style: const TextStyle(
fontSize: 23, color: Colors.white, fontWeight: FontWeight.w900),
),
],
),
),
));
}

// Create list to store the refs :
final List<DatabaseReference> refs = [
  FirebaseDatabase.instance.ref().child('mq9/value'),
  FirebaseDatabase.instance.ref().child('temp/value'),
  FirebaseDatabase.instance.ref().child('humidity/value')
];
```

References

- [1]: Amazon web service.**
- [2]: Flutter Pub.dev.**
- [3]: Firebase Website.**
- [4]: Mouser Electronics.**
- [5]: Sparkfun.**
- [6]: Components101.**
- [7]: ST Company.**
- [8]: UElectronics.**